

Assignment 3: Industrial AI for Wear Estimation: A feature-based approach

Reference paper: Najeh, Lundberg, Kerrouche (2021). *Deep-Learning and Vibration-Based System for Wear Size Estimation of Railway Switches and Crossings*. *Sensors*, 21, 5217.

1. Background (read paper Sections 1 to 3 first)

Railway Switches & Crossings (S&C) wear out overtime. Direct inspection (cameras, manual tools) is expensive. The paper proposes an indirect approach: install accelerometers inside the point machine and let a deep-learning model estimate the wear level from the vibration signal of a bogie passing over the S&C.

In this assignment you will work on the middle section (S1S2) of the test rig and classify the wear into four levels.

Target labels (4 classes): L0 (no wear), L1 (1st level), L2 (2nd level), L3 (3rd level), see Table 4 of the paper for the actual wear depths.

2. The data

- **Sampling rate:** 51.2 kHz (vibration), 1 Hz (speed).
- **Run length:** 120 seconds per pass.
- **Sensors:** several accelerometers in the point machine (paper Fig. 6, Table 1). For this tutorial use **one channel** (e.g. accelerometer A, 10 kHz range, X direction).
- **Crop:** keep only the segment where the bogie is over **S1S2 = 8 m** (the middle section, paper Fig. 10).
- **Windowing:** split each pass into **40 windows of 3 s**. Each 3-s window contains $3 \times 51200 = 153\,600$ samples.

Each DTMS file has columns: `time`, `ax`, `ay`, `az`, `speed`.

3. Method: LSTM on time-domain features

Goal

Train an LSTM that classifies the wear level (L0/L1/L2/L3) from the **time-domain features** of one 3-s vibration window.

Step 1: Extract 7 features per block

Reshape every 3-s window into **256 blocks of 600 samples** (paper §3.3.1). For each block compute:

#	Feature	Formula
1	Root Mean Square (RMS)	$\sqrt{(1/N) * \sum(x_i^2)}$
2	Skewness	$\sum((x_i - m)^3) / ((N-1) * \sigma^3)$
3	Kurtosis	$\sum((x_i - m)^4) / ((N-1) * \sigma^4)$
4	Shape factor	$\text{RMS} / \text{mean}(x)$
5	Crest factor	$\max(x) / \text{RMS}$
6	Impulse factor	$\max(x) / \text{mean}(x)$
7	Clearance (margin)	$\max(x) / (\text{mean}(\sqrt{ x }))^2$

Result per window: a tensor of shape **(256 time-steps, 7 features)**.

Step 2: Build the LSTM

Step 3: Train / test split

- 60 % training, 40 % testing (same as paper).
- Epochs = 100, batch size = 32.

Step 4: Evaluate

- Plot the **confusion matrix** (4×4) compare with paper Fig. 12 (paper reports ≈ 88 % average accuracy).
- Compute per-class accuracy.

What to hand in

- Code (notebook or script).
- Trained model weights.
- Confusion matrix figure (4×4) and overall accuracy.
- Short discussion (1 page): which classes are confused? Why does L1 vs L2 perform worse than L0 vs L3? (Hint: read paper §4.3.)